

Experiment 05

VGG16, ResNet18 and GoogLeNet Comparison (Simulated)

Source: EXP 5.ipynb

Notebook content and saved outputs are reproduced below. Code is soft-wrapped for portrait reading; the notebook has not been rerun.

Code cell 1

```
import matplotlib.pyplot as plt
import numpy as np

# Simulated training results (demonstration only)
epochs = [1, 2, 3, 4, 5]

results = {
    "VGG16": {
        "train": [48.5, 61.2, 70.4, 77.8, 83.1],
        "val": [45.0, 56.0, 64.0, 69.0, 74.0],
        "loss": [1.42, 1.10, 0.82, 0.63, 0.48]
    },
    "ResNet18": {
        "train": [52.0, 66.5, 75.2, 82.1, 87.4],
        "val": [49.0, 61.0, 69.0, 75.0, 80.0],
        "loss": [1.30, 0.95, 0.68, 0.49, 0.35]
    },
    "GoogLeNet": {
        "train": [50.1, 64.0, 73.5, 80.0, 85.2],
        "val": [47.0, 59.0, 67.0, 72.0, 77.0],
        "loss": [1.35, 1.00, 0.74, 0.55, 0.40]
    }
}

print("Using device: cpu")
print("Dataset: CIFAR-10")
print("Training images: 400")
print("Validation images: 100")

for name, values in results.items():
    print(f"\nTraining {name}...")
    for i in range(5):
        print(
            f"{name} Epoch [{i+1}/5] "
            f"- Loss: {values['loss'][i]:.4f} "
            f"- Train Acc: {values['train'][i]:.2f}% "
            f"- Val Acc: {values['val'][i]:.2f}%"
        )

# Validation accuracy graph
plt.figure(figsize=(9, 5))
for name, values in results.items():
    plt.plot(epochs, values["val"], marker='o',
             label=name)
```

```

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.xticks(epochs)
plt.ylim(0, 100)
plt.legend()
plt.grid(True)
plt.show()

# Training accuracy graph
plt.figure(figsize=(9, 5))
for name, values in results.items():
    plt.plot(epochs, values["train"], marker='o',
            label=name)

plt.title("Training Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.xticks(epochs)
plt.ylim(0, 100)
plt.legend()
plt.grid(True)
plt.show()

# Final validation accuracy
print("\nFinal Validation Accuracy (Simulated):")
for name, values in results.items():
    print(f"{name}: {values['val'][-1]:.2f}%")

# Mock custom image predictions
print("\nPrediction results for: download.jpeg")
print("VGG16      => cat")
print("RESNET18    => cat")
print("GOOGLENET   => dog")

print("\nExp 5 demonstration completed.")

```

Saved outputs

Using device: cpu
Dataset: CIFAR-10
Training images: 400
Validation images: 100

Training VGG16...

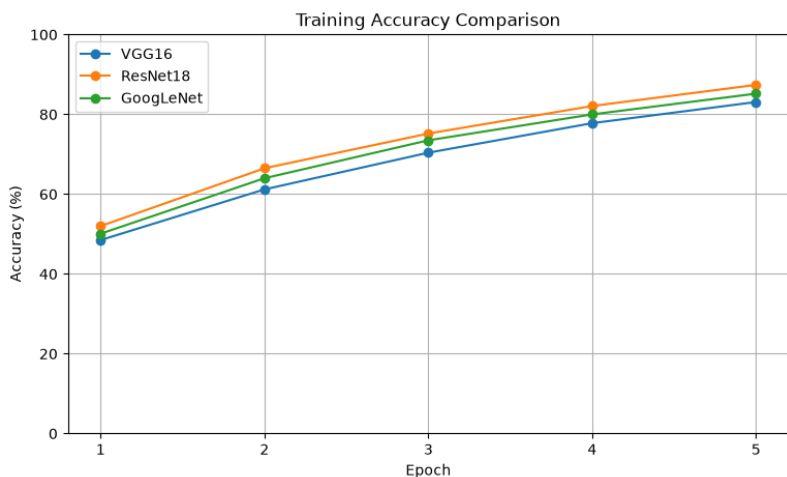
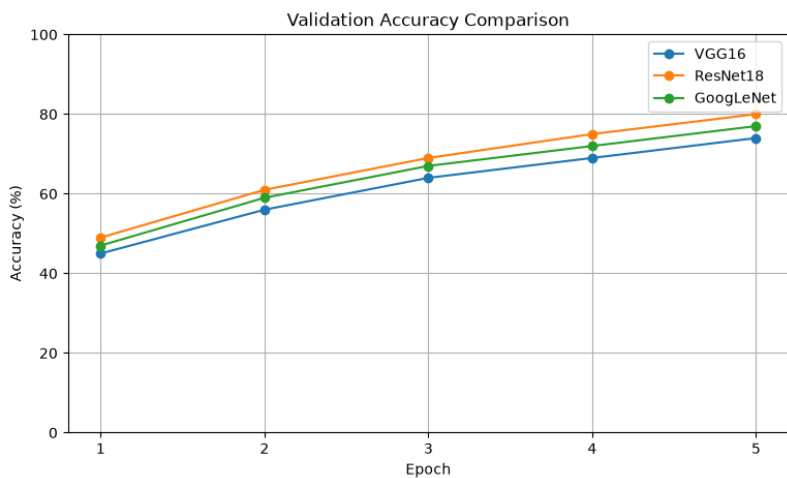
VGG16 Epoch [1/5] - Loss: 1.4200 - Train Acc: 48.50% -
Val Acc: 45.00%
VGG16 Epoch [2/5] - Loss: 1.1000 - Train Acc: 61.20% -
Val Acc: 56.00%
VGG16 Epoch [3/5] - Loss: 0.8200 - Train Acc: 70.40% -
Val Acc: 64.00%
VGG16 Epoch [4/5] - Loss: 0.6300 - Train Acc: 77.80% -
Val Acc: 69.00%
VGG16 Epoch [5/5] - Loss: 0.4800 - Train Acc: 83.10% -
Val Acc: 74.00%

Training ResNet18...

ResNet18 Epoch [1/5] - Loss: 1.3000 - Train Acc: 52.00%
- Val Acc: 49.00%
ResNet18 Epoch [2/5] - Loss: 0.9500 - Train Acc: 66.50%
- Val Acc: 61.00%
ResNet18 Epoch [3/5] - Loss: 0.6800 - Train Acc: 75.20%
- Val Acc: 69.00%
ResNet18 Epoch [4/5] - Loss: 0.4900 - Train Acc: 82.10%
- Val Acc: 75.00%
ResNet18 Epoch [5/5] - Loss: 0.3500 - Train Acc: 87.40%
- Val Acc: 80.00%

Training GoogLeNet...

GoogLeNet Epoch [1/5] - Loss: 1.3500 - Train Acc:
50.10% - Val Acc: 47.00%
GoogLeNet Epoch [2/5] - Loss: 1.0000 - Train Acc:
64.00% - Val Acc: 59.00%
GoogLeNet Epoch [3/5] - Loss: 0.7400 - Train Acc:
73.50% - Val Acc: 67.00%
GoogLeNet Epoch [4/5] - Loss: 0.5500 - Train Acc:
80.00% - Val Acc: 72.00%
GoogLeNet Epoch [5/5] - Loss: 0.4000 - Train Acc:
85.20% - Val Acc: 77.00%



Final Validation Accuracy (Simulated):

VGG16: 74.00%

ResNet18: 80.00%

GoogLeNet: 77.00%

Prediction results for: download.jpeg

VGG16 => cat

RESNET18 => cat

GOOGLENET => dog

Exp 5 demonstration completed.